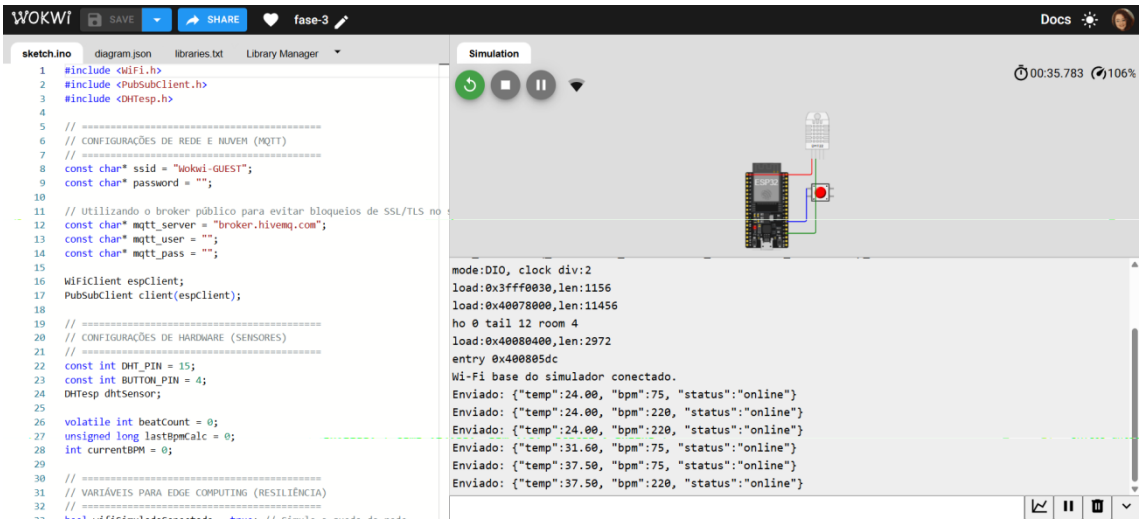
Relatório Técnico — Fluxo de Funcionamento e Lógica de Resiliência (Edge Computing)

1. Introdução e Arquitetura de Hardware na Borda (Edge): O subsistema do projeto Cardio IA foi desenvolvido utilizando o microcontrolador ESP32 no simulador Wokwi para monitorizar sinais vitais de forma autônoma. A arquitetura integra um sensor digital de temperatura e umidade DHT22 (pino GPIO 15) e um botão de pressão (Push button no pino GPIO 4) para simulação de pulsos cardíacos. O pino do botão utiliza resistor de *pull-up* interno e interrupção de hardware (IRAM_ATTR countBeat) na borda de descida (FALLING), garantindo contabilização precisa dos batimentos diretamente na memória RAM em microssegundos.

2. Modelagem Fisiológica e Execução: O firmware opera de forma não bloqueante via millis(). A cada janela de 5 segundos, o sistema calcula a frequência cardíaca partindo de uma base de repouso de 75 BPM, somando 12 BPM por clique (75 + (beatCount * 12)). Uma trava biológica restringe o envio ao teto máximo de **220 BPM**, evitando ruídos mecânicos (*bounce*). Paralelamente, o sensor DHT22 fornece a leitura térmica em graus Celsius.

3. Lógica de Resiliência e Tolerância a Falhas: Para evitar a perda de dados clínicos em quedas de rede, a borda conta com um buffer local em memória RAM (struct Dado) com capacidade para até 20 registros. No modo online, os dados são publicados via MQTT em formato JSON com a flag "status":"online". Se a rede cair (comando Serial 0), as leituras são armazenadas localmente no buffer. Ao restaurar a rede (comando Serial 1), o sistema esvazia o buffer transmitindo os registros históricos com a flag "status":"recuperado", assegurando integridade total da telemetria.


WOKWI
SAVE
SHARE
fase-3
Docs
sketch.ino
diagram.json
libraries.txt
Library Manager
Simulation
00:35.783
106%
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHTesp.h>
// ==
// CONFIGURAÇÕES DE REDE E NUVEM (MQTT)
// ==
const char* ssid = "Wokwi-GUEST";
const char* password = "";
// Utilizando o broker público para evitar bloqueios de SSL/TLS no
const char* mqtt_server = "broker.hivemq.com";
const char* mqtt_user = "";
const char* mqtt_pass = "";
WiFiClient espClient;
PubSubClient client(espClient);
// ==
// CONFIGURAÇÕES DE HARDWARE (SENSORES)
// ==
const int DHT_PIN = 15;
const int BUTTON_PIN = 4;
DHTesp dhtSensor;
volatile int beatCount = 0;
unsigned long lastBpmCalc = 0;
int currentBPM = 0;
// ==
// VARIÁVEIS PARA EDGE COMPUTING (RESILIÊNCIA)
// ==
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Wi-Fi base do simulador conectado.
Enviado: {"temp":24.00, "bpm":75, "status":"online"}
Enviado: {"temp":24.00, "bpm":220, "status":"online"}
Enviado: {"temp":24.00, "bpm":220, "status":"online"}
Enviado: {"temp":31.60, "bpm":75, "status":"online"}
Enviado: {"temp":37.50, "bpm":75, "status":"online"}
Enviado: {"temp":37.50, "bpm":220, "status":"online"}